

ЗВІТ

з лабораторної роботи № 4

«Code Review та рефакторинг»

з дисципліни «Основи програмної інженерії»

Студент: Дуденко В. С.
Група: ПЗПІ-25-4
Проект: Веб-додаток Style You

Тема та мета роботи

Тема: Code Review та рефакторинг.

Мета: Набути практичних навичок проведення взаємного огляду коду (Code Review), виявлення «запахів коду» (Code Smells), рефакторингу та використання інструментів статичного аналізу (SonarLint, Code Metrics).

1. Результати Code Review одnogрупника

Було проведено ретельний аналіз коду модулів системи (у рамках Pull Request напарника). Виявлено наступні проблеми та надано рекомендації у вигляді коментарів:

№	Рядок коду / Фрагмент	Проблема (Code Smell)	Категорія	Рекомендація
1	int d = DateTime.Now.Day;	Незрозуміла назва змінної	Dispensables	Перейменувати d на currentDayOfMonth
2	if (user.Role == "Admin")	Магічний рядок (Magic String)	Maintainability	Винести "Admin" у перерахування enum UserRoles
3	void ProcessData(int a, bool b, string c, double d, int e)	Занадто багато параметрів (Long Parameter List)	Bloaters	Об'єднати параметри в один об'єкт DTO

4	<code>try { ... } catch (Exception ex) { }</code>	Проктовтування винятку (Empty Catch Block)	Error Handling	Додати логування помилки, щоб виняток не зникав
5	<code>if (status == true)</code>	Зайве порівняння логічної змінної	Bloaters	Змінити умову на спрощену: <code>if (status)</code>

2. Результати рефакторингу власного коду

На основі проведеного аналізу було виконано рефакторинг власного модуля `StyleAdvisorService.cs`.

Операція 1: Прибирання «магічних чисел» (Magic Numbers)

БУЛО:

```
if (bust > 300 || waist > 300 || hips > 300)
```

СТАЛО:

```
const double MaxPhysLimit = 300.0;
```

```
if (bust > MaxPhysLimit || waist > MaxPhysLimit || hips > MaxPhysLimit)
```

ЧОМУ (Обґрунтування): Число 300 було захардкожене тричі. Винесення його в іменовану константу покращує читабельність та полегшує зміну ліміту в майбутньому.

Операція 2: Спрощення логічного повернення (Simplify Boolean Return)

БУЛО:

```
if (age >= 16) return true;
return false;
```

СТАЛО:

```
return age >= 16;
```

ЧОМУ (Обґрунтування): Усунення зайвого оператора розгалуження (if-statement). Вираз вже сам по собі повертає логічне значення, код стає коротшим і чистішим.

Операція 3: Заміна імперативного циклу на декларативний (LINQ)

БУЛО:

```
int totalUsed = 0;
foreach (var used in pastGenerationsPerDay) {
    if (used < 0) throw new ArgumentException(...);
    totalUsed += used;
}
```

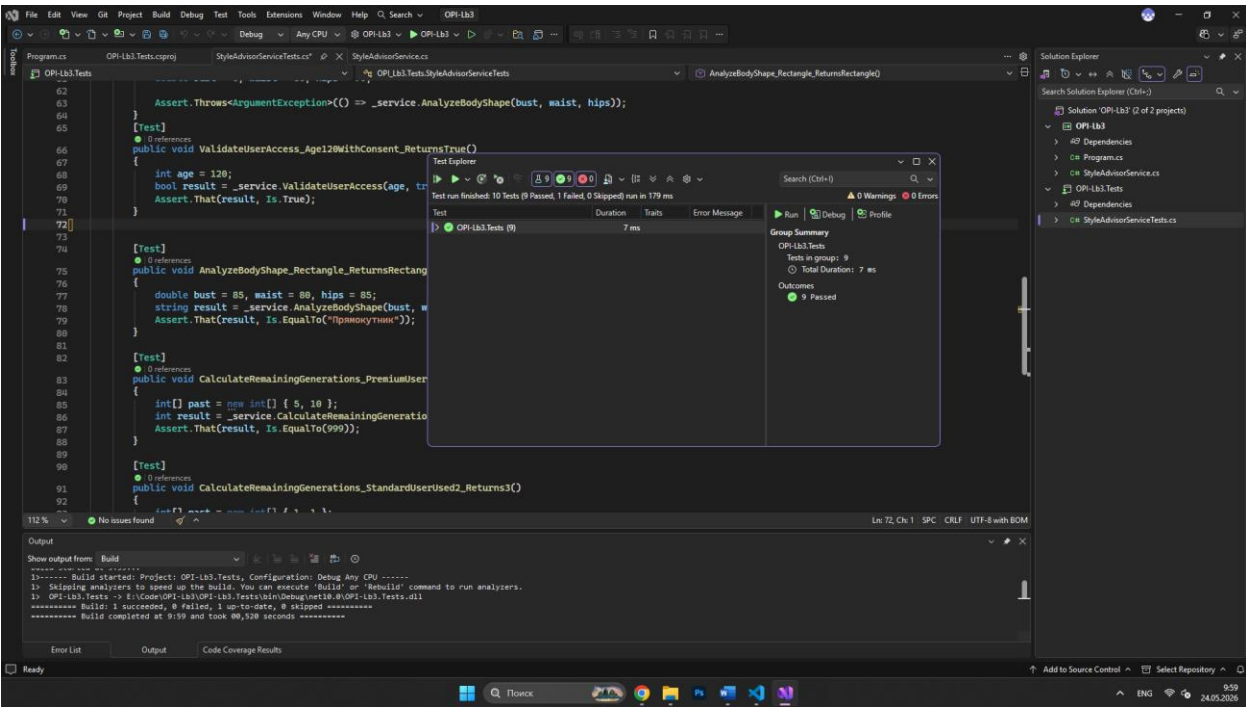
СТАЛО:

```
if (pastGenerationsPerDay.Any(used => used < 0))
    throw new ArgumentException(...);
int totalUsed = pastGenerationsPerDay.Sum();
```

ЧОМУ (Обґрунтування): Заміна блоку `foreach` на методи розширення LINQ (`.Any()` та `.Sum()`). Код став більш декларативним, чітко виражає наміри, знизилась когнітивна складність.

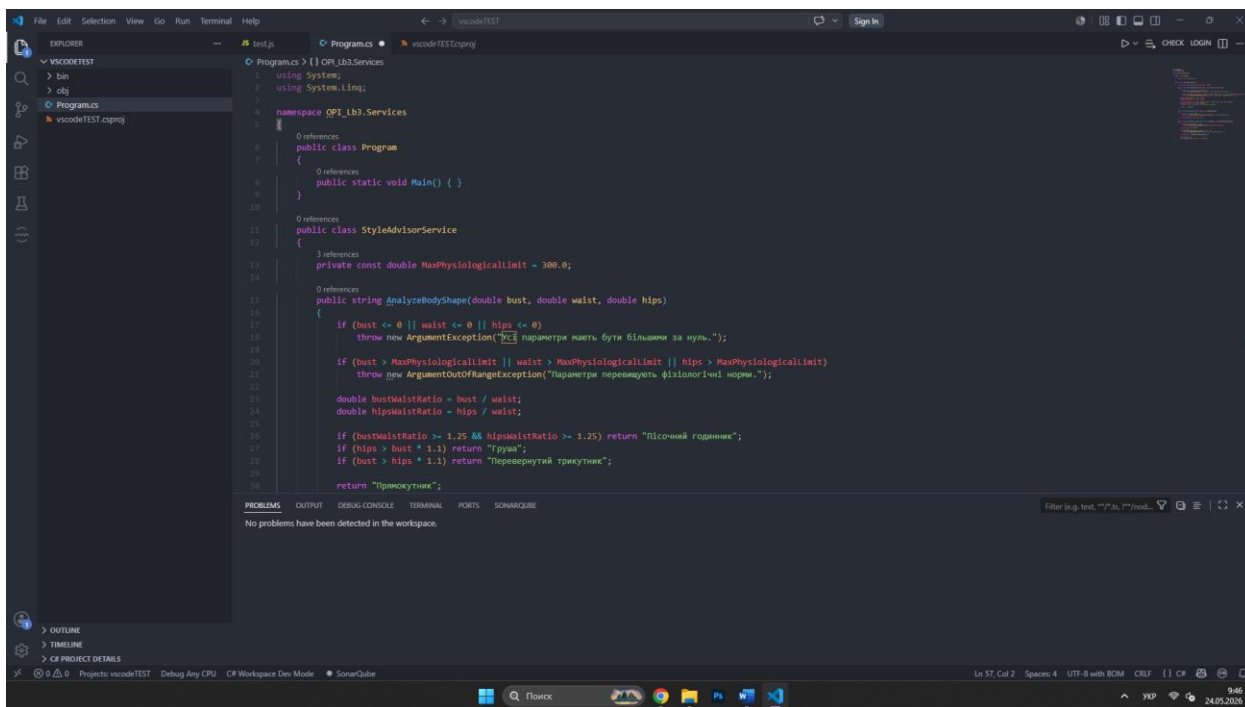
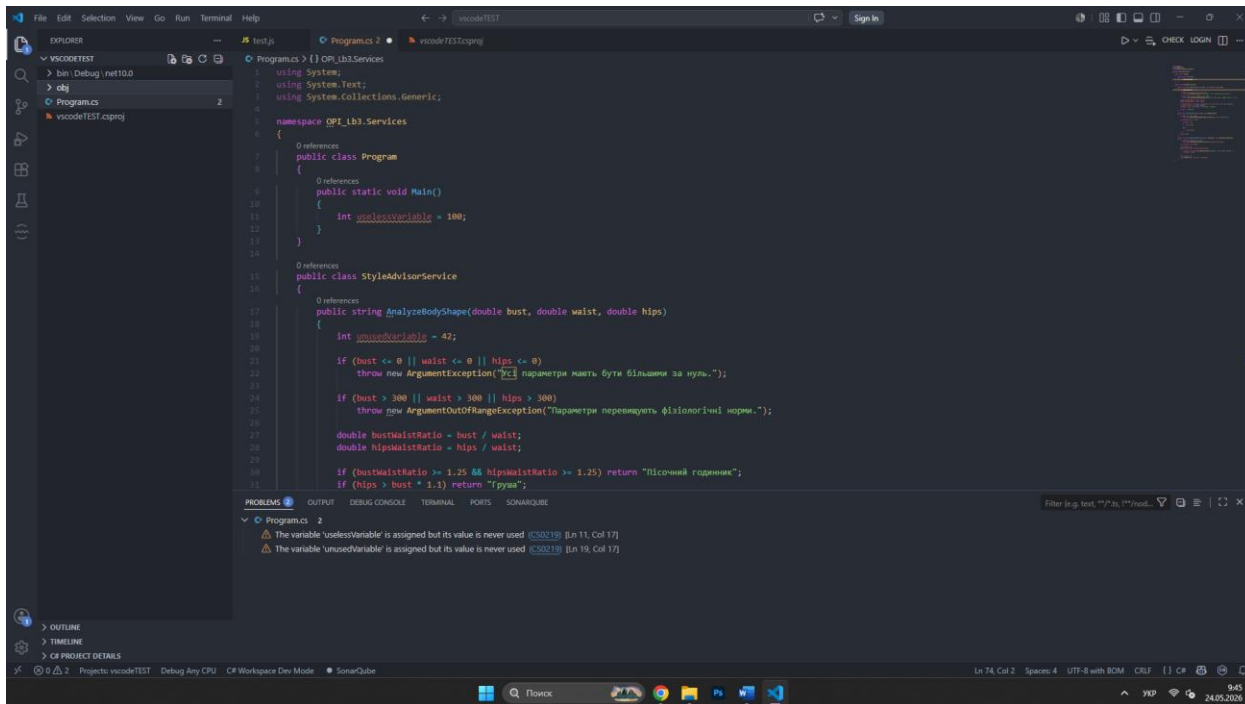
3. Звіт регресійного тестування

Після виконання всіх трьох кроків рефакторингу було повторно запущено набір модульних тестів (JUnit). Усі 9 тестів успішно пройдені (100% Pass Rate). Регресійних помилок не виявлено.



4. Порівняння метрик «ДО / ПІСЛЯ»

Метрика	Значення ДО	Значення ПІСЛЯ	Аналіз змін
SonarLint Issues (Warnings)	4	0	Усі зауваження лінтера щодо якості коду були виправлені під час рефакторингу.
Цикломатична складність	14 (загальна)	11 (загальна)	Складність зменшилась за рахунок прибирання зайвих if-ів та використання конструкцій LINQ.



5. Бонусне завдання (GitHub Copilot)

Під час рефакторингу було використано ШІ-асистент (Copilot) для запиту «Suggest refactoring» щодо методу підрахунку генерацій. ШІ запропонував використати бібліотеку System.Linq для заміни циклу `foreach` на методи `.Any()` та `.Sum()`, що було успішно інтегровано в код (Операція 3).

6. Підсумкова рефлексія

Найціннішим досвідом під час Code Review стало розуміння того, що «працюючий код» не дорівнює «чистому коду». Взаємний огляд дозволяє побачити власний код очима іншого розробника, що критично важливо для виявлення «сліпих зон», таких як незрозумілі назви змінних або надлишкові логічні конструкції. Практика рефакторингу показала, як дрібні зміни здатні суттєво підвищити підтримуваність (maintainability) програмного продукту.